FELADATKIÍRÁS

Ennek az oldalnak a helyére a feladatkiírást kell beszúrnod az elektronikus változatban (praktikusan egy képernyőkép formájában). A nyomtatott változatban ennek az oldalnak a helyére a diplomaterv portálról letöltött, jóváhagyott feladatkiírást kell befűzni. A feladatkiírás tanszéki aláírás és pecsét nélkül is érvényes.

Budapesti Műszaki és Gazdaságtudományi Egyetem
Villamosmérnöki és Informatikai Kar
Automatizálási és Alkalmazott Informatikai Tanszék

Horváth Gellért

AI-TÁMOGATOTT TÁRGYFELISMERÉS ÉS HASONLÓSÁGVIZSGÁLAT MOBILALKALMAZÁSBAN

KONZULENS

DR. Ekler Péter

BUDAPEST, 2025

Tartalomjegyzék

Összefoglaló **5**
Abstract **6**
1 Bevezetés **7**
2 Alkalmazott technológiák **10**
3 Architektúra ..... Hiba! A könyvjelző nem létezik.
4 Projekt részletezése **14**
5 Projektértékelés **15**
6 Összefoglaló **16**
7 Irodalomjegyzék ..... Hiba! A könyvjelző nem létezik.

HALLGATÓI NYILATKOZAT

Alulírott **Horváth Gellért** szigorló hallgató kijelentem, hogy ezt a szakdolgozatot meg nem engedett segítség nélkül, saját magam készítettem, csak a megadott forrásokat (szakirodalom, eszközök stb.) használtam fel. Minden olyan részt, melyet szó szerint, vagy azonos értelemben, de átfogalmazva más forrásból átvettem, egyértelműen, a forrás megadásával megjelöltem.

Hozzájárulok, hogy a jelen munkám alapadatait (szerző, cím, angol és magyar nyelvű tartalmi kivonat, készítés éve, konzulens(ek) neve) a BME VIK nyilvánosan hozzáférhető elektronikus formában, a munka teljes szövegét pedig az egyetem belső hálózatán keresztül (vagy hitelesített felhasználók számára) közzétegye. Kijelentem, hogy a benyújtott munka és annak elektronikus verziója megegyezik. Dékáni engedéllyel titkosított diplomatervek esetén a dolgozat szövege csak 3 év eltelte után válik hozzáférhetővé.

Kelt: Budapest, 2025. 10. 19

...…….

Horváth Gellért

Összefoglaló

A szakdolgozat, vagy diplomaterv elkészítése minden egyetemi hallgató életében egy fontos mérföldkő. Lehetőséget ad arra, hogy az egyetemi évei során megtanultakat kamatoztassa és eredményeit szélesebb közönség előtt bemutassa, s mérnöki rátermettségét bizonyítsa. Fontos azonban, hogy a dolgozat elkészítésének folyamata számos csapdát is rejt magában. Rossz időgazdálkodás, hiányos szövegszerkesztési ismeretek, illetve a dolgozat készítéséhez nélkülözhetetlen „műfaji" szabályok ismeretének hiánya könnyen oda vezethetnek, hogy egy egyébként jelentős időbefektetéssel készült kiemelkedő szoftver is csak gyengébb minősítést kapjon a gyenge minőségű dolgozat miatt.

E dokumentum – amellett, hogy egy általános szerkesztési keretet ad a dolgozatodnak – összefoglalja a szakdolgozat/diplomaterv írás írott és íratlan szabályait. Összeszedjük a Word kezelésének legfontosabb részeit (címsorok, ábrák, irodalomjegyzék stb.), a dolgozat felépítésének általános tartalmi és szerkezeti irányelveit. Bár mindenkire igazítható sablon természetesen nem létezik, megadjuk azokat az általános arányokat, oldalszámokat, amelyek betartásával jó eséllyel készíthetsz egy színvonalas dolgozatot. A részletes és pontokba szedett elvárás-lista nem csupán a dolgozat írásakor, de akár más dolgozatok értékelésekor is kiváló támpontként szolgálhat.

Az itt átadott ismeretek és szemléletmód nem csupán az aktuális feladatod leküzdésében segíthet, de hosszútávon is számos praktikus fogással bővítheti a szövegszerkesztési és dokumentumkészítési eszköztáradat.

Abstract

Ide jön a ½-1 oldalas angol nyelvű összefoglaló, amelynek szövege a Diplomaterv Portálra külön is feltöltésre kerül. Ez a magyar nyelvű összefoglaló angolra fordított változata.

1 Bevezetés

Egy szakdolgozat projekt mindig a feladat megálmodásával kezdődik. Ez a „beugró” rész sok hallgatónál a teljes projekt egyik legnehezebb része lehet, ugyanis a hallgatónak meg kell találnia a számos téma közül azt, amin szívesen dolgozik, végig fenntartja az érdeklődését, továbbá objektív szemléletet is alkalmaz, vagyis igyekszik olyan témát választani aminek helye lehet a modern szoftverpiacon, új dolgokat tanul belőle, viszont nem vállalja túl magát. Ez a döntés rendkívül fontos, ennek terhe a teljes féléven végig kísér.

Számomra a téma kiválasztása egyszerű volt, segített benne az, hogy egyből találtam hozzá személyes, illetve a szakmámhoz köthető motivációt is. A személyes motivációm kiindulópontja a hobbim volt. Hobbi szinten gyűjtöm az üveges üdítők és sörök fémkupakjait. Nem tudom, hogy más gyűjtemények tulajdonosai hogyan tartják számon a több száz darabból álló kollekcióikat, de nekem ez igen nehezen ment. Sokszor már a megvétel előtt sem tudtam eldönteni, melyik üdítőt vagy sört válasszam – az elfogyasztása mellett azzal a céllal, hogy az új fémkupak bekerüljön a gyűjteményembe. Lényegében ez a probléma volt az, ami a személyes inspirációt adta a téma kiválasztásához. A személyes motivációm ismertetése már utalhatott arra, hogy a projektem egy kollekció, gyűjteménynyilvántartó rendszer megvalósítása. Eredetileg az alkalmazást kizárólag a kupakgyűjtemények kezelésére terveztem, azonban a konzulensemmel egyeztetve arra jutottunk, hogy az alkalmazás szélesebb felhasználói bázist érhetne el, ha univerzális megoldást kínálna. Ennek megfelelően a feladatot kibővítettem, így az új cél egy általános kollekciókezelő rendszer megvalósítása lett. Fontos tisztázni, mit jelent az „univerzális megoldás”. Ez azt jelenti, hogy a felhasználók a kollekció létrehozásakor többféle kollekciótípus közül választhatnak, amelyeket a rendszer teljes körűen támogat. Emellett új kollekciótípusok támogatása is gyorsan és egyszerűen hozzáadható a programhoz anélkül, hogy a meglévő kódot módosítani kellene. Bár a program alapvető célja már világos, eddig semmi rendkívüli nincs benne: a leírás akár egy hagyományos könyvtárkezelő alkalmazásra is vonatkozhatna.

Az én kupak gyűjteményem írásban nehezen dokumentálható. Ezt egy példán keresztül szemléltetem: egy kupakot sokkal nehezebb virtuális kollekcióba felcímkézni és adatokkal ellátni, mint egy fociskártyát. A fociskártyák általában egyediek: egy

játékoshoz egy lap tartozik, vagy esetenként több, amelyek ritkasági kategóriákba vannak sorolva. A lényeg az, hogy 1-2 tulajdonság alapján egyértelműen beazonosíthatók, és ezeket a tulajdonságokat maga a kártya is egyértelműen megmutatja. Egy kupak ezzel szemben csak több tulajdonság alapján azonosítható pontosan — ilyen például a márka, a márkán belüli típus, az esetleges évjárat vagy egy különkiadás. A beazonosítást tovább nehezíti, hogy ezek az adatok sok kupakon nincsenek egyértelműen feltüntetve, vagyis nem olvashatók le közvetlenül — ezek kiderítése további utánajárást igényel. Pontosan ezért, számomra, és más gyűjtők számára, akiknek a gyűjteményébe a tárgyak nehezen dokumentálhatóak, egy egyszerű könyvtár alkalmazás nem lenne nagy segítség. Valahogy máshogy szerettem volna dokumentálni a tárgyakat. Kitaláltam, hogy a tárgyak fő tulajdonsága egy fotó legyen róluk. Ez minden korábban említett problémát megold, hiszen egyrészt ez az egy tulajdonság egyértelműen beazonosítja a tárgyat, másrészt manapság egy telefonnal nagyon egyszerűen készíthetünk jó minőségű fényképeket. Így az AI sokkal gyorsabban tudja ellenőrizni, majd mi felvenni az új tárgyakat a kollekcióba, mintha a nehezen kivehető szöveges adatok között kellene keresgélnünk. Természetesen a program úgy lett tervezve, hogy támogassa a felcímkézést is, tehát a képek alapján felvett tárgyakat később adatokkal is kiegészíthetjük. Az előbb már meg is említettem a szakmához köthető motivációkat, az alábbiakban ezeket fejtem ki: az AI és az okostelefonok. Napjaink modern és exponenciálisan fejlődő világában egyre nagyobb teret hódít az AI és annak forradalmi megoldásai, míg az okostelefonok továbbra is a legnépszerűbb elektronikus eszközök közé tartoznak. Egy szakmabelinek ezért elengedhetetlen, hogy legalább alapszinten ismerje ezeket a technológiákat. Ennek tudatában választottam meg a szakmámhoz köthető motivációt. A telefonra való integráció kézenfekvő volt, hiszen a tárgyakról telefonnal lehet fotót készíteni, és ezeket a képeket azonnal továbbítani lehet a telefonon futó alkalmazásnak. Az AI technológia alkalmazása pedig azért vált szükségessé, mert a képet valamilyen módon le kell szűkíteni magára a tárgyra, illetve két fotót meg kell tudni különböztetni egymástól, hogy megállapíthassuk, egy tárgy egyedi-e a kollekcióban. Ezt a funkciót mesterséges intelligencia segítségével lehet megvalósítani. A mesterséges intelligencián alapuló tárgyfelismerés és az egyediségvizsgálat képezi az alkalmazás központi funkcióját; a szakdolgozatom címében szereplő „tárgyfelismerés és hasonlóságvizsgálat” is ezt tükrözi.

Egy valós telefonos alkalmazás rengeteg funkcióval rendelkezik. Én is egy ilyen, valós alkalmazás modellezését tűztem ki célul. Ezért a feladathoz néhány közösségi funkciót is hozzáadtam, hogy a rendszer egy teljes értékű mobilalkalmazást szimuláljon. A plusz funkciók a közösségre épülnek: a felhasználók egy „feed” oldalon láthatják mások új szerzeményeit, feliratkozhatnak más gyűjteményekre, amelyek külön oldalon jelennek meg, valamint a „feed” elsősorban az azonos országból származó bejegyzéseket és a követett kollekciók új tárgyait jeleníti meg. Ezen felül helyet kapott egy csere-bere funkció is, ahol a felhasználók csereajánlatokat tölthetnek fel - akár több tárgyat is egy bejegyzésbe -, és más felhasználók ezekre ajánlatokat küldhetnek, amelyek közül a bejegyzés feltöltője válogathat.

Összefoglalva a motivációk mentén tagolt feladatleírásomat: A projekt célja egy univerzális kollekció- és gyűjteménykezelő rendszer megalkotása, amelynek fő funkciója a tárgyak egyediségének vizsgálata tárgyfelismerésen és hasonlóságvizsgálaton keresztül. A rendszer támogatja a kollekciók létrehozását és szerkesztését, valamint kiegészül közösségi funkciókkal is, mint a „feed” oldal, a kollekciók követése és a csere-bere felület. A teljes projekt így egy komplex, közösségi élményt nyújtó mobilplatformot modellez.

Mostanra tisztán látható, mi a projekt célja, és milyen motivációk vezettek a feladat meghatározásához. Az alábbiakban bemutatom a teljes projektet. Először sorra veszem az alkalmazott technológiákat, a legegyszerűbben megérthetőktől a legösszetettebbek felé haladva. Ezt követően bemutatom az alkalmazás alapjául szolgáló architektúrát, amelyre a backend szerverek épülnek. Ismertetem, miként sikerült elérni az univerzalitást a különböző kollekciótípusok támogatásában. Ezután végigvezetem az olvasót a teljes projekten, és bemutatom annak legfontosabb részeit – mind a backend, mind a frontend oldaláról. A dolgozat végén értékelem a munkámat: megvizsgálom, milyen minőségben sikerült megvalósítani a program komponenseit, és milyen eredményt hozott a fő funkció implementálása. A szakdolgozatot egy rövid összegzéssel zárom, amelyben összefoglalom, hogy a feladatkiírásból mit valósítottam meg, milyen irányokban lehetne továbbfejleszteni a rendszert, mit tanultam a folyamat során, valamint milyen pozitív és negatív tapasztalatokat szereztem a szoftverfejlesztés ezen területén.

2 Alkalmazott technológiák

2.1 PostgreSQL

Minden alkalmazás alapját egy adatbázis képezi, ahova a felhasználók által generált adatok menthetők. Az adatbázisoknak számos követelménynek kell megfelelniük: biztosítaniuk kell az adatbiztonságot és az adatvédelmet, hibamentesen kell működniük, valamint támogatniuk kell a többfelhasználós környezetet.

Napjainkban rengeteg lehetőség közül választhatunk - léteznek relációs és NoSQL adatbázisok is. Bár a NoSQL megoldások egyre nagyobb népszerűségnek örvendenek, én a relációs adatbázis mellett döntöttem, mivel főként ehhez tanultam meg az adatstruktúra-tervezés alapjait. A relációs adatbázisok nagy része hasonló elven működik, csak kisebb különbségek figyelhetők meg köztük - például bizonyos SQL-funkciók elérhetőségében. A választást általában nem ezek a különbségek, hanem sokkal inkább a fejlesztői tapasztalat, a használt eszközök ismerete, illetve a projektbe illeszthetőség határozza meg. Én is ezen a szemléleten haladtam, és azt a megoldást választottam, amellyel korábban már dolgoztam: így az adatbázisom PostgreSQL alapokon fut.

2.2 ASP.NET Core (backend)

Az applikáció backend szerverét ASP.NET Core technológiával valósítottam meg. Azért választottam ezt a keretrendszert, mert a C# nyelvre épül, amelyet már ismerek, és egyetemi tanulmányaim során is foglalkoztam az ASP.NET Core-ral – habár nem ilyen mélységben, mint ennél a projektnél. A döntésben az is szerepet játszott, hogy ez az egyik leggyakrabban használt technológia weboldalak és alkalmazások backend szervereként, amihez számos segédanyag és dokumentáció érhető el, így elakadás esetén gyorsan találhatók megoldások.

A Microsoft által fejlesztett ASP.NET keretrendszer-család frontend- és backend-fejlesztéshez egyaránt kínál megoldásokat. Én ebből a családból a Core verziót használtam, amely több korábbi ASP.NET keretrendszer előnyeit egyesíti egy modern, egységes keretrendszerben. Az ASP.NET Core platformfüggetlen, nagy teljesítményű, és számos hasznos szerveroldali funkciót kínál. Ezek közül a projekt során elsősorban az API-fejlesztési lehetőségeit használtam ki, mivel minimális konfigurációval is gyorsan

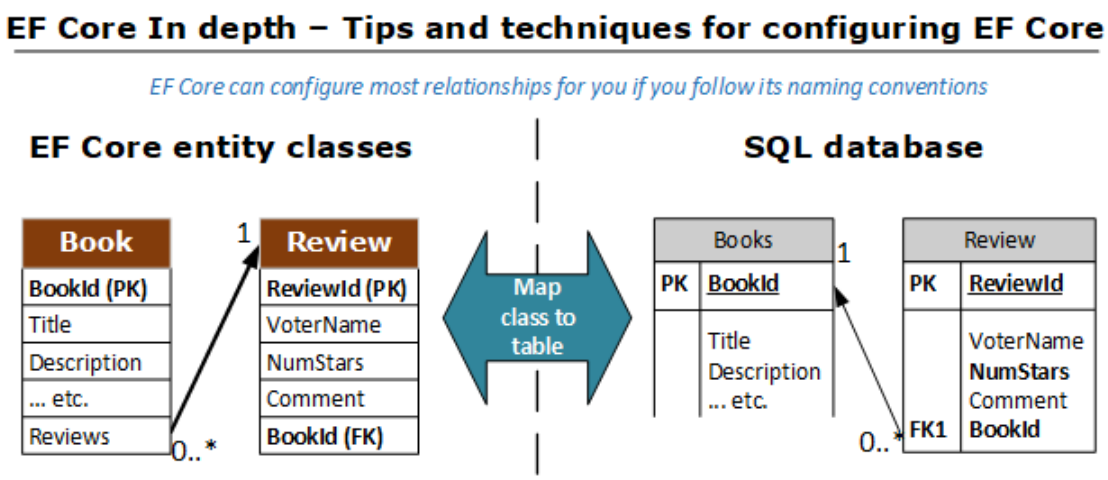
létrehozható egy jól működő szerveroldali API. A frontend mobilalkalmazás ezen az API-n keresztül éri el az adatbázis adatait CRUD műveletekkel, valamint a bonyolultabb use case-ekhez külön, célzott végpontokat is kialakítottam. Alkalmazásfejlesztés során mindig fontos szempont, hogy a rendszer bármely komponense később lecserélhető legyen más implementációra. Ezt interfészek használatával lehet megvalósítani, azonban minden interfészhez meg kell adnunk egy implementációt is, ami nagyobb projektekben bonyolulttá válhat, ha a függőségeket nem kezeljük egységesen. Ezért a fejlesztés során tudatosan nem csatoltam közvetlenül konkrét implementációkat a komponensekhez, hanem csak interfészeket használtam, és a függőségek kezelésére függőség-injektálást alkalmaztam. A függőséginjektálás egy összetettebb koncepció, amelyet nem biztos, hogy mindenki részletesen ismer vagy ért. A Medium oldalon található ismertető [1] egy könnyen érthető magyarázatot nyújt a fogalom megértéséhez. Az ASP.NET Core egyik nagy előnye, hogy ehhez beépített, teljes körű támogatást biztosít, ami jelentősen megkönnyíti a fejlesztést és a kód karbantarthatóságát. Az ASP.NET Core további előnyeiről a Microsoft hivatalos honlapján olvashatunk. [2]

2.3 Entity Framework

A Microsoft egyik jelentős fejlesztése az Entity Framework Core, amely eredetileg Entity Framework néven indult, és kezdetben kizárólag objektum-relációs leképezést (ORM) biztosított. Az új, továbbfejlesztett verzió - az Entity Framework Core - ezt a funkcionalitást számos új képességgel bővítette, például a platformfüggetlenséggel. Bár ez a keretrendszer nem része közvetlenül az ASP.NET Core-nak, a kettőt gyakran együtt használják. Én is mindkettőt alkalmaztam a projekt során. Számomra az Entity Framework Core legfontosabb funkciói az objektum-relációs leképezés, valamint a navigációs tulajdonságok voltak.

2.3.1 Objektum-relációs leképezés

Az objektum-relációs leképezést (ORM) megvalósító könyvtárak automatikusan elvégzik a konverziót a relációs adatbázis táblái és az objektum-orientált programozási nyelv osztályai között. Ez azt jelenti, hogy ha ilyen könyvtárat használunk, nem kell külön logikát írnunk ahhoz, hogy a táblákból érkező rekordokat a megfelelő objektumokhoz rendeljük, mert ezt a könyvtár automatikusan kezeli. A megfeleltetés fordított irányban is működik: adatíráskor az objektum alapján a neki megfelelő rekord kerül be az adatbázis táblájába.



1. ábra Példa objektum-relációs leképezésre [3]

A leképezés lényegét az 1. ábra látható példán keresztül szeretném bemutatni. Az SQL-adatbázisban két tábla található: *Books* és *Review*. Ehhez létrehozunk két C# osztályt, amelyeket tetszőlegesen elnevezhetünk. A lényeg az, hogy az osztály tulajdonságai az aktuális névkonvenciót kövessék. A névkonvenció határozza meg, hogy az adatbázis-tábla mezőihez mely C# osztálytulajdonságok rendelődnek hozzá. A példában a PascalCase konvenció van érvényben, így a könyvtár tudja, hogy a *BookId* C# tulajdonság a tábla *BookId* mezőjéhez tartozik. Minden ORM-könyvtár rendelkezik egy alapértelmezett névkonvencióval, amely szükség esetén módosítható.

A név konvenciós szabályok:

- Camel Case: firstName
- Snake Case: first_name
- Kebab Case: first-name
- Pascal Case: FirstName

```
public DbSet<Book> Books { get; set; }
```

2. ábra Táblázat reprezentáció Entity Framework Core-ban

A táblázat nevénél is figyelni kell a névkonvencióra. A táblázat Entity Framework Core-ban, *DbSet*-ként van reprezentálva, itt a változó neve a névkonvenció alapján meg kell egyezzen a táblázat nevével.

2.3.2 Navigációs tulajdonságok

A navigációs tulajdonságok rendkívül hasznosak: segítségükkel az adatbázisban valamilyen asszociációban álló táblák között, a C#-ban reprezentált objektumok szintjén könnyedén navigálhatunk, anélkül hogy a kapcsolótáblát külön le kellene kérdeznünk és manuálisan összekapcsolnunk az adatait. Ez azt jelenti, hogy egy osztályban tulajdonságokkal jelölhetjük a táblák közötti kapcsolatokat, így a kapcsolat másik oldalán lévő C#-objektumok bármely elemét közvetlenül elérhetjük. A 2.3.1-es pontban ismertetett példánál maradva: ez lehetővé teszi például, hogy egy könyv objektumon keresztül közvetlenül elérjük a hozzá tartozó értékeléseket, vagy fordítva, egy értékelésből visszahivatkozzunk a kapcsolódó könyvre.

3 Projekt részletezése

4 Projektértékelés

5 Összefoglaló

Irodalomjegyzék

[1] Dev Cookies, „The Ultimate Guide to Dependency Injection: Everything You Need to Know," 18 október 2024. [Online]. Available: https://devcookies.medium.com/the-ultimate-guide-to-dependency-injection-everything-you-need-to-know-7dfea8178739. [Hozzáférés dátuma: 6 november 2025].

[2] Microsoft, „Overview of ASP.NET Core," 2025. [Online]. Available: https://learn.microsoft.com/en-us/aspnet/core/overview?view=aspnetcore-9.0. [Hozzáférés dátuma: 6 november 2025].

[3] A. P. Smith, „The Reformed Programmer," [Online]. Available: https://www.thereformedprogrammer.net/wp-content/uploads/2020/07/ConfigurationHeader.png. [Hozzáférés dátuma: 6 november 2025].